ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería de Sistemas · Carrera de Ciencia de Datos e Inteligencia Artificial

I N F O R M E D E L A B O R A T O R I O · S E M A N A 3

Clasificación de variables y población vs. muestra

Laboratorio complementario · Semana 3

Asignatura IDSD422 — Estadística y Programación para Ciencia de Datos I

Estudiante(s)	[GOMEZ GUAMAN AITANA RAQUEL] · usuario de GitHub: [argg1006]
Docente	[CHRISTIAN SUAREZ]
Período académico	2026-B
Fecha de entrega	[23/10/2026]
Repositorio (fork)	https://github.com/argg1006/IDSD422-lab-muestreo-poblacion.git

1 · Introducción

En el presente trabajo se analiza el caso de estudio de la empresa RetailAndina S. A., cuya gerencia requiere analizar a profundidad su base de datos de fidelización, la cual está compuesta por 400 clientes, con el propósito de comprender la naturaleza de sus variables y estimar de forma precisa el ingreso mensual promedio de sus clientes a partir de muestras reducidas. Para llevarlo a cabo, el estudio se divide en dos fases técnicas: primero, la clasificación tipológica y correcta declaración de las variables en Python y R, de esta forma previniendo errores analítica graves que derriben de un tipado incorrecto, como la falta de orden en variables ordinales. Segundo, la implementación y contraste de tres diseños de muestreo como lo son: conveniencia, aleatorio simple y estratificado, los cuales serán evaluados directamente contra el parámetro poblacional real. Está práctica resulta necesaria para la ciencia de datos, debido a que demuestra que el análisis estadístico riguroso depende directamente de una adecuada estructuración de los datos y de un diseño muestral idóneo, evidenciando que los sesgos de selección no se corrigen simplemente aumentando el tamaño de la mejores, sino mediante una serie de criterios probabilísticos y de estratificación que garanticen una representatividad de la población.

2 · Objetivos

3 · Desarrollo de la práctica

Para el desarrollo de la práctica se usó Google colab, usamos dos lenguajes de programación, en este caso R y Pyton, es importante recordar que se debe cargar la base de datos en cada uso de Google colab, caso contrario no se podrá ejecutar los comandos obtenidos.

Dentro del desarrollo de la practica no alcanzara la evidencia total en crudo ya que ocupa espacio que es necesario, para ello solo se mostrara los resultados.

Cabe recalcar que para que se lea R debemos usar %load_ext rpy2.ipython, de esta manera se puede usar el lenguaje R dentro de Google colab, que es una extensión conectada a una librería para su uso.

Clasificación variable.py

```
print(f"  {variable:20s} {tipo}")
```

```
Dimensiones: 12 filas x 8 columnas

Primeras filas:
cliente_id  edad  numero_hijos    genero nivel_educacion    ciudad  ingresos_mensuales  satisfaccion
      C001    34             2  Femenino     Universidad     Quito                1450             4
      C002    27             0 Masculino      Secundaria Guayaquil                 980             3
      C003    41             3  Femenino       Posgrado    Cuenca                2200             5
      C004    22             0 Masculino     Universidad     Quito                 760             2
      C005    36             1  Femenino      Secundaria Guayaquil                1150             3
      C006    29             2      Otro     Universidad     Quito                1350             4
      C007    48             4 Masculino       Primaria    Cuenca                 890             2
      C008    33             1  Femenino     Universidad Guayaquil                1680             4
      C009    25             0 Masculino      Secundaria     Quito                 840             3
      C010    39             2  Femenino       Posgrado Guayaquil                2450             5
      C011    31             1      Otro     Universidad    Cuenca                1210             4
      C012    45             3 Masculino      Secundaria     Quito                1070             2

Tipos inferidos (dtype):
cliente_id            object
edad                   int64
numero_hijos           int64
genero                object
nivel_educacion       object
ciudad                object
ingresos_mensuales     int64
satisfaccion           int64
dtype: object

Variable cliente_id | Únicos: 12 | Valores: ['C001', 'C002', 'C003', 'C004', 'C005', 'C006', 'C007', 'C008', 'C009', 'C010', 'C011', 'C012']
Variable edad | Únicos: 12 | Valores: [np.int64(34), np.int64(27), np.int64(41), np.int64(22), np.int64(36), np.int64(29), np.int64(48), np.int64(33), np.int64(25), np.int64(39), np.int64(31), np.int64(45)]
Variable numero_hijos | Únicos: 5 | Valores: [np.int64(2), np.int64(0), np.int64(3), np.int64(1), np.int64(4)]
Variable genero | Únicos: 3 | Valores: ['Femenino', 'Masculino', 'Otro']
Variable nivel_educacion | Únicos: 4 | Valores: ['Universidad', 'Secundaria', 'Posgrado', 'Primaria']
Variable ciudad | Únicos: 3 | Valores: ['Quito', 'Guayaquil', 'Cuenca']
Variable ingresos_mensuales | Únicos: 12 | Valores: [np.int64(1450), np.int64(980), np.int64(2200), np.int64(760), np.int64(1150), np.int64(1350), np.int64(890), np.int64(1680), np.int64(840), np.int64(2450), n
Variable satisfaccion | Únicos: 4 | Valores: [np.int64(4), np.int64(3), np.int64(5), np.int64(2)]
Primaria
Faltantes con typo: 7
Resumen de la clasificación (para el informe):
  cliente_id          Cualitativa nominal
  edad                Cuantitativa discreta
  numero_hijos        Cuantitativa discreta
  genero              Categórica binaria
  nivel_educacion     Categórica ordinal
  ciudad              Categórica nominal
  ingresos_mensuales  Cuantitativa continua
  satisfaccion        Categórica ordinal
```

Explicación del código: Se carga la base de datos con el uso de pandas y luego se lee el archivo, luego usamos el shape para sacar las dimensiones y dtype para sacar el tipo de variable, luego se usa la función unique para

detectar valores como el genero que termina siendo dos variables binarias y variables nominales. Finalmente usa la declaración de orden pd.CategoricalDtype para tener en orden las variables ordinales.

Clasificación variable.py

```
Dimensiones: 12 filas × 8 columnas
   cliente_id edad numero_hijos    genero nivel_educacion    ciudad
1        C001   34            2  Femenino     Universidad     Quito
2        C002   27            0 Masculino      Secundaria Guayaquil
3        C003   41            3  Femenino        Posgrado    Cuenca
4        C004   22            0 Masculino     Universidad     Quito
5        C005   36            1  Femenino      Secundaria Guayaquil
6        C006   29            2      Otro     Universidad     Quito
7        C007   48            4 Masculino        Primaria    Cuenca
8        C008   33            1  Femenino     Universidad Guayaquil
9        C009   25            0 Masculino      Secundaria     Quito
10       C010   39            2  Femenino        Posgrado Guayaquil
11       C011   31            1      Otro     Universidad    Cuenca
12       C012   45            3 Masculino      Secundaria     Quito
   ingresos_mensuales satisfaccion
1                1450            4
2                 980            3
3                2200            5
4                 760            2
5                1150            3
6                1350            4
7                 890            2
8                1680            4
9                 840            3
10               2450            5
11               1210            4
12               1070            2
'data.frame':   12 obs. of  8 variables:
 $ cliente_id       : chr  "C001" "C002" "C003" "C004" ...
 $ edad             : int  34 27 41 22 36 29 48 33 25 39 ...
 $ numero_hijos     : int  2 0 3 0 1 2 4 1 0 2 ...
 $ genero           : chr  "Femenino" "Masculino" "Femenino" "Masculino" ...
 $ nivel_educacion  : chr  "Universidad" "Secundaria" "Posgrado" "Universidad" ...
 $ ciudad           : chr  "Quito" "Guayaquil" "Cuenca" "Quito" ...
 $ ingresos_mensuales: int  1450 980 2200 760 1150 1350 890 1680 840 2450 ...
 $ satisfaccion     : int  4 3 5 2 3 4 2 4 3 5 ...
Variable cliente_id | Únicos: 12 | Valores: C001, C002, C003, C004, C005, C006, C007, C008, C009, C010, C011, C012
Variable edad | Únicos: 12 | Valores: 34, 27, 41, 22, 36, 29, 48, 33, 25, 39, 31, 45
Variable numero_hijos | Únicos: 5 | Valores: 2, 0, 3, 1, 4
Variable genero | Únicos: 3 | Valores: Femenino, Masculino, Otro
Variable nivel_educacion | Únicos: 4 | Valores: Universidad, Secundaria, Posgrado, Primaria
Variable ciudad | Únicos: 3 | Valores: Quito, Guayaquil, Cuenca
Variable ingresos_mensuales | Únicos: 12 | Valores: 1450, 980, 2200, 760, 1150, 1350, 890, 1680, 840, 2450, 1210, 1070
Variable satisfaccion | Únicos: 4 | Valores: 4, 3, 5, 2
[1] <NA>
Levels: Primaria < Secundaria < Superior < Posgrado
Faltantes con typo: 7

Resumen de la clasificación (para el informe):
  cliente_id         Cualitativa nominal
  edad               Cuantitativa discreta
  numero_hijos       Cuantitativa discreta
  genero             Categórica binaria
  nivel_educacion    Categórica ordinal
  ciudad             Categórica nominal
  ingresos_mensuales Cuantitativa continua
  satisfaccion       Categórica ordinal
```

Explicación del código: Se parece bastante al código usado en Python, pero con diferencia en cuanto se trata de las funciones, en este caso para las variables usamos str(), para variables únicas se revisa con unique, en este caso se usa una combinación con length(), para variables ordinales se usa factor(……ordered=TRUE).

Población vs muestra.py

```
Población: 400 clientes × 8 columnas
Parámetro poblacional — ingreso medio: 1330.72 USD
Ciudades en la población: {'Quito': 200, 'Guayaquil': 120, 'Cuenca': 80}

[conveniencia] n=20  media=1348.50 USD  (parámetro 1330.72, diferencia +17.78)
    composición por ciudad: {'Quito': 20}
[Conveniencia] n=20  media=1348.50 USD  (parámetro 1330.72, diferencia +17.78)
    composición por ciudad: {'Quito': 20}
[Aleatorio simple] n=20  media=1336.00 USD  (parámetro 1330.72, diferencia +5.28)
    composición por ciudad: {'Cuenca': 8, 'Quito': 8, 'Guayaquil': 4}
[Estratificado] n=20  media=1287.50 USD  (parámetro 1330.72, diferencia -43.22)
    composición por ciudad: {'Quito': 10, 'Guayaquil': 6, 'Cuenca': 4}

Parámetro poblacional: 1330.72 USD
                         diseño  n  media estimada
           Conveniencia (head) 20          1348.5
     Aleatorio simple (sample) 20          1336.0
Estratificado (groupby + frac) 20          1287.5
```

Explicación del código: Se lee la base y calcula el ingreso promedio de los 400 clientes. Para los ejemplos de muestreo empezamos con el de conveniencia, que se toma los 20 primero que se encuentran en archivo usando el .head(20), luego el de aleatorio, en este caso se usa la función sample(20), es aleatorio entre 20 clientes, finalmente el estratificado, lo agrupamos con groupby y un sample, en este caso lo que hace es dividir por estratos, en este caso las ciudades y luego saca una muestra proporcional por cada grupo.

Población vs muestra.R

```
Población: 400 clientes × 8 columnas
Parámetro poblacional — ingreso medio: 1330.72 USD
Ciudades en la población:
   Cuenca Guayaquil     Quito
       80       120       200
[Conveniencia] n=20  media=1348.50 USD  (parámetro 1330.72, diferencia +17.78)
    composición por ciudad:
Quito
   20
[Aleatorio simple] n=20  media=1318.00 USD  (parámetro 1330.72, diferencia -12.72)
    composición por ciudad:
   Cuenca Guayaquil     Quito
        5         7         8
[Estratificado] n=20  media=1554.00 USD  (parámetro 1330.72, diferencia +223.28)
    composición por ciudad:
   Cuenca Guayaquil     Quito
        4         6        10
Parámetro poblacional: 1330.72 USD
                      diseno  n media_estimada
          Conveniencia (head) 20         1348.5
    Aleatorio simple (sample) 20         1318.0
 Estratificado (slice_sample) 20         1554.0
```

Explicación del código: Al igual que en Python en este caso cargamos el archivo y lo leemos, en este caso seria los 400 clientes, sacando la media o promedio de esta. Luego empezamos con el muestreo que en este caso seria por conveniencia, para ello usamos igual el head para los primeros 20 clientes, para el aleatorio simple, tenemos que usar el sample, y fijamos una semillla set.seed, para que el resultado sea correcto, y finalmente para el estratificado, usamos una librería extra que es dplyr, para agrupar los datos por ciudad y luego extraemos de acuerdo al estrato, con slice_simple, con un 5%, asegura cuotas específicas.

Variable	Tipo	Justificación (criterio)	Verificación en código

cliente_id	Cualitativa nominal	Funciona puramente como etiqueta de identificación	Object en (py), chr(R)
edad	Cuantitativa discreta	Cantidad numérica exacta	Int64 en (py), int (R)
numero_hijos	Cuantitativa discreta	Cantidad numérica exacta	Int64 en (py), int (R)
genero	Categórica nominal	Agrupa los registros en categorías cualitativas sin orden	Object en (py), chr(R)
nivel_educacion	Categórica ordinal	Posee una jerarquía	Object en (py), chr(R)
ciudad	Categórica nominal	Agrupa registros por la ubicación geográfica sin orden	Object en (py), chr(R)
ingresos_mensuales	Cuantitativa continua	Admite infinitos valores	Int64 en (py), int (R
satisfaccion	Categoría ordinal	Mide en una escala valorativa	Int64 en (py), int (R

P1. ¿Qué te dijo el dtype de satisfaccion y por qué no basta para clasificarla? ¿Qué pasa si una ordinal queda como texto sin declarar el orden? (Responde con la evidencia de tu ejecución.)

3.2 · Parte B · Población vs. muestra

- La población es data/clientes_400.csv : **400 clientes** (Quito 200 / Guayaquil 120 / Cuenca 80). La variable de interés es ingresos_mensuales .
- Primero calcula el **parámetro poblacional** y completa la tabla. Luego corre los tres diseños y registra sus salidas **tal como salieron**. La composición por ciudad es la pista que delata el sesgo.

Concepto	Valor
Población (N)	400 clientes
Variable de interés	Ingresos_mensuales
Parámetro poblacional (media)	1,330.72
¿Por qué se muestrea si aquí el parámetro existe y se calculó?	Se puede calcular el parámetro exacto porque ya están determinado los 400 clientes, pero en una practica real requiere tiempo

Diseño	n	Media estimada	Composición por ciudad	Diferencia vs. parámetro
Conveniencia (head)	20	1348.5	Quito:20	+17.78

Aleatorio simple (sample)	20	1318	Cuenca: 8, Quito: 8, Guayaquil: 4	5.28
Estratificado (groupby + 5 %)	20	1287.5	Quito: 10, Guayaquil: 6, Cuenca: 4	-43.22

Anota también las medias de 5 sorteos distintos del aleatorio simple (semillas 1 a 5):

Semilla	1	2	3	4	5
Media	1336	1405	1235	1491.5	1288

P2. ¿Cuánto vale el parámetro poblacional y cuánto estimó cada diseño? ¿Cuál quedó más cerca y por qué eso no alcanza para declarar un ganador?

Ya se muestran en la tabla los valores de cada diseño, pero el poblacional es de 1330.72. En este caso la muestra aleatoria simple es la que estuvo más cerca, pero si se cambia la semilla y se extrae otra muestra cambia el diseño. Se debe valor por sus propiedades teóricas, además tomando en cuenta que no debe haber sesgos.

P3. La muestra por conveniencia también tiene 20 filas. ¿Qué defecto tiene y cómo lo delatas en la composición por ciudad? ¿Se corrige tomando más filas?

Presenta un sesgo de selección severa, al no otorgar las mismas oportunidades a las diferentes filas. y no al aumentar el tamaño de una muestra sesgada no soluciona el problema, solo lo solidifica, ampliando el error sistemático.

P4. El sorteo aleatorio dio un número distinto en cada semilla. ¿Qué implica esa variabilidad para una empresa que va a decidir con un solo estudio?

Implica la existencia de incertidumbre y un error de muestreo. Conlleva un error porque no podemos estar repitiendo el estudio, por esto el análisis real debe tomar en cuenta diferentes cosas, como tener intervalos de confianza o márgenes de error.

P5. ¿Cuándo usarías cada diseño en la práctica? Relaciona cada uno con un uso real (piloto, subgrupos que deben aparecer, conclusión sobre toda la población).

Muestreo por conveniencia: Se usa en fases exploratorias, no se debe usar para inferir conclusiones estadísticas.

Muestreo aleatorio simple: Se emplea cuando la población es relativamente homogénea, además de que deben tener una probabilidad de cada elemento por así decirlo ser seleccionado. Busca emitir conclusiones especificas sin requerir control forzoso sobre subgrupos específicos.

Muestreo estratificado: Se selecciona cuando la población esta compuesta por subgrupos, en este caso puede ser geográficos, o sucursales, por ejemplo, que deben aparecer representados en proporciones adecuadas.

4 · ¿Qué aprendí con esta práctica?

Ahora entiendo como funciona los tipos de datos, sobre todo si bien entiendo que son enteros y characters u objetos por así decirlo, verlo de forma concreta de cuantitativa o cualitativo, a si como su respectiva agrupación,

de ordinal o nominal, o discreta o indiscreta, me parece importante sobre todo para la estadística poder diferencias estas agrupaciones.

A su vez también como existen diferentes maneras de hacer una muestra, como lo son por conveniencia que termina siendo la más rápida, pero nada eficaz, luego el aleatorio siempre, que busca mejorar los fallos de la conveniencia, pero igual termina siendo algo ineficaz porque no tomamos en cuenta otros factores, y finalmente la estratificada, que para mi gusto personal es la más adecuada en este caso, ya que no solo ve una variable en sí, si no varias.

En cuento a codificación siempre se aprende algo nuevo con Python y R, sobre todo R, que es algo que no había visto antes, pero sobre todo las similitudes en estos, tiene un lenguaje parecido, lo que facilita en entendimiento de los códigos.

5 · Conclusiones

- Se clasificaron y verificaron exitosamente las 8 variables de la base de datos de fidelización de *RetailAndina S.A.* con los 400 clientes. evidenciando mediante código que el tipo informático (dtype / class) es insuficiente para definir la naturaleza estadística de un atributo.
- Al contrastar tres diseños de muestreo n=20 ,frente al parámetro poblacional real del ingreso medio de 1330.72, se comprobó empíricamente que el muestreo por conveniencia (head), demostrando que un error sistemático no se corrige aumentando el tamaño de la muestra.

6 · Bibliografía

GeeksforGeeks. (s.f.). *Python random.sample() function*. GeeksforGeeks. Recuperado el 24 de octubre de 2026, de https://www.geeksforgeeks.org/python/python-random-sample-function/

Webster, W. (2023, 30 de enero). *Sampling methods, types & techniques*. Qualtrics. https://www.qualtrics.com/articles/strategy-research/sampling-methods/